Bitcoin Quantum: A Post-Quantum Bitcoin Model

BTQ Technologies

August 7, 2026

Abstract

Bitcoin Quantum (BTQ) is a UTXO-model cryptocurrency whose transaction authorization need not rest on the one cryptographic assumption a quantum computer is known to break. BTQ is a codebase fork of Bitcoin Core (`btq-ag/btq-core`) operating as a separate, replay-isolated blockchain. It ships ML-DSA-44 (Dilithium2, NIST FIPS 204) as its signature scheme, implements BIP-360 Pay-to-Merkle-Root (P2MR) for the address and script system, and retains SHA-256 proof of work. Signature schemes enter the protocol through a defined lifecycle — opcode-family allocation gated by height-based activation — and ML-DSA-44 is the first scheme deployed through it. Introducing a successor scheme requires consensus implementation and activation, but not a redesign of the UTXO or Script execution model; generic multi-scheme address and script encoding is future work. The urgency of retiring ECDSA is quantitative: recent resource estimates put secp256k1 key recovery as low as 1,200 logical qubits and 90 million Toffoli gates under a fast-clock attack model, an order of magnitude below prior published estimates. We describe the protocol and its security properties under a quantum adversary.

1 Introduction

Bitcoin's security rests on the elliptic curve discrete logarithm problem (ECDLP) over secp256k1 [11], an assumption Shor's algorithm breaks in polynomial time on a sufficiently large quantum computer. Bitcoin Quantum (BTQ) retires that assumption while preserving the rest of Bitcoin's design: it ships ML-DSA-44 (Dilithium2, NIST FIPS 204) [12] as its signature scheme, together with the consensus changes required to carry it, and leaves the UTXO model, Script execution semantics, and SHA-256 proof of work in place. BTQ is a codebase fork of Bitcoin Core (`btq-ag/btq-core`) operating as a separate, replay-isolated blockchain. BTQ is not a Bitcoin replacement, not an attempt to migrate the Bitcoin ledger, and not a soft- or hard-fork of the Bitcoin chain itself. The migration path for the Bitcoin chain itself involves social consensus, economic coordination, and backward-compatibility constraints outside the scope of this paper.

The planning horizon for the post-quantum transition is set by attack-resource estimates, and those estimates have recently contracted by an order of magnitude. Babbush et al. (April 2026) establish resource estimates for the ECDLP attack via a zero-knowledge-verified quantum circuit: breaking secp256k1 requires either $\leq$1,200 logical qubits and $\leq$90 million Toffoli gates, or $\leq$1,450 logical qubits and $\leq$70 million Toffoli gates, depending on which resource is prioritized [3]. On a superconducting surface-code architecture with 10^{-3} physical gate error and planar connectivity, these circuits execute on fewer than half a million physical qubits. This is about an order of magnitude smaller in both qubit count and gate cost than the best prior published estimates. At 10-microsecond control-system reaction times, 70 or 90 million Toffoli gates resolve in 18 or 23 minutes respectively. That places key recovery inside the typical Bitcoin block confirmation

window: an on-spend attack derives the private key from an exposed public key after transaction broadcast and before confirmation, and fast-clock hardware makes that attack relevant to Bitcoin's confirmation model. Babbush et al. conclude that vulnerable cryptocurrency communities should begin the PQC transition immediately, before the remaining migration window narrows further [3]. Bitcoin Quantum is a concrete instantiation of that transition on a Bitcoin-derived codebase.

2 Quantum threats to Bitcoin

2.1 Architectures and attack speed

Cryptographically relevant quantum computers (CRQCs) that could execute Shor's algorithm against the elliptic-curve discrete logarithm problem (ECDLP) that underlies secp256k1 fall into two broad hardware classes, distinguished primarily by gate clock rate and error-correction cycle time [3].

Fast-clock CRQCs include superconducting, silicon spin, and photonic architectures. They operate with control-system reaction times on the order of 10 microseconds. At that cadence, 70 or 90 million Toffoli gates resolve in 18 or 23 minutes respectively on a superconducting surface-code architecture at 10^{-3} physical gate error [3]. Because a CRQC can precompute the first half of Shor's algorithm offline, it can enter a "primed" state. That first half depends only on fixed protocol parameters. The machine need only complete the second half once a target public key appears. From that primed state, Babbush et al. estimate that first-generation fast-clock CRQCs can solve ECDLP on secp256k1 in about 9 minutes on average [3]. Bitcoin's average block time is 10 minutes. An on-spend attack recovers the key during the window between transaction broadcast and block confirmation. That window is within the fast-clock estimate. On a typical single-signature Bitcoin transaction, Babbush et al. give a 41% probability of theft before the victim's transaction is confirmed [3].

Slow-clock CRQCs include neutral-atom and trapped-ion architectures. They operate two to three orders of magnitude more slowly. Babbush Figure 14 places per-key derivation time for slow-clock devices in the range of 14 hours to 12 days [3]. These devices cannot meet the mempool confirmation window required for on-spend attacks, but they are fully capable of *at-rest* attacks: deriving private keys from public keys that are permanently exposed on-chain, given sufficient time.

Shor's algorithm is the underlying primitive in both cases [14, 3]. The attack class that a specific CRQC can execute depends on whether its clock speed permits key recovery within Bitcoin's confirmation window. The exposed vulnerability class depends on which public keys are visible to the attacker and when. Attack speed is one half of the threat model. The other half is which Bitcoin outputs expose public keys, and by what mechanism.

2.2 Vulnerability classes

The taxonomy of Bitcoin quantum vulnerabilities was originated by Aggarwal et al. (2018) [1] and quantitatively refined by Babbush et al. (2026) [3]. Four distinct exposure classes arise from the different ways a public key can become known to a quantum attacker.

Weak Address. Certain script types record the public key directly in the locking script, permanently and visibly on-chain from the moment funds are received. P2PK outputs (used extensively in the Satoshi-era coinbase and early transactions), legacy P2MS multisignature, and P2TR (Taproot, introduced in 2021) all fall into this category. A Weak Address is exposed to both at-rest attacks

by slow-clock CRQCs and on-spend attacks by fast-clock CRQCs from the time the receiving transaction is confirmed.

Address Reuse. Script types that hash the public key, including P2PKH, P2SH, P2WPKH, and P2WSH, conceal it until the first spend. At that point the unlocking script reveals the full public key on-chain. Any UTXO remaining at that address after the first spend is therefore exposed for at-rest attack by any CRQC that indexes the chain. Address reuse converts an otherwise hash-protected address into a Weak Address for all residual funds, permanently.

Public Mempool Exposure. Bitcoin spends reveal the signature material needed to validate the input before any miner includes the transaction in a block. Across the deployed script types, that means essentially every spend exposes the public key or script material from which the relevant public key is obtained during validation. The exposure persists until the transaction is confirmed. A fast-clock CRQC monitoring the mempool can attempt key derivation during this window. Stewart et al. (2018) identified this surface and proposed a commit-delay-reveal construction as a mitigation [15]. Absent a protocol-level fix, the surface remains open for all current Bitcoin users.

Offchain Exposure. Public keys and extended public keys (xpubs) propagate beyond the chain itself through wallet software, portfolio tracking services, cross-chain bridges, and exchange custody systems. An attacker with access to any of these repositories can mount at-rest attacks against the associated addresses without requiring any on-chain observation. The same is true for an attacker who can observe xpub-derived key reuse across chains. This class arises from user practices and ecosystem conventions rather than protocol-level design choices, making it the hardest to bound or eliminate through protocol changes alone.

2.3 The Bitcoin exposure profile

Applying the vulnerability taxonomy to the current Bitcoin UTXO set yields a concrete exposure profile. Project Eleven's Risq List estimates about 6.9 million BTC locked in addresses with directly derivable or historically exposed public keys: about 1.72 million BTC in P2PK outputs (Weak Address, including Satoshi-era mining rewards) and about 4.99 million BTC across address-reused P2PKH and related script types [13]. Babbush Figures 5 and 7 provide per-address and aggregate balance distributions corroborating this scale [3]. Table 1 summarises the exposure profile of each Bitcoin script type, mapping it to the vulnerability classes it incurs and the share of supply at risk.

Script type	Weak Addr	Reuse	Mempool	Offchain	Vulnerable supply
P2PK	✓	—	✓	✓	∼1.72 M BTC (Satoshi-era)
P2PKH	—	✓	✓	✓	via address reuse
P2MS	✓	—	✓	✓	legacy multisig (small share)
P2SH	—	✓	✓	✓	via address reuse
P2WPKH	—	✓	✓	✓	via address reuse
P2WSH	—	✓	✓	✓	via address reuse
P2TR	✓	—	✓	✓	most-recent script type

Table 1: Bitcoin script-type exposure to quantum attacks. Adapted from Babbush et al. Table I [3] and the Project Eleven Risq List [13]. P2MR (proposed; not yet a Bitcoin standard script type) would eliminate Weak Address. Total Bitcoin supply at risk of at-rest attack is about 6.9 M BTC per the Risq List methodology.

Beyond these static holdings, Public Mempool Exposure applies to essentially all Bitcoin spends that

reveal signature material, regardless of script type or address hygiene. Every transaction broadcast creates a window in which a sufficiently fast CRQC can attempt key derivation before confirmation. This near-universal exposure is the primary motivator for post-quantum signature adoption at the protocol level: even a user who rotates addresses on every transaction and holds no P2PK outputs incurs mempool exposure on every spend. No address-hygiene practice eliminates this surface. Only a replacement signature scheme does.

2.4 Proof of work is not the main risk

A persistent misconception frames Grover's algorithm as a threat to Bitcoin's SHA-256 proof-of-work, potentially enabling quantum-accelerated mining that could destabilize the network. Babbush et al. address this directly [3]: Grover's algorithm provides only a quadratic speedup for unstructured search problems, compared to Shor's exponential speedup for ECDLP [9]. The quadratic advantage is largely consumed by quantum error-correction overhead: fault-tolerant execution of Grover's circuit requires a large number of error-correction cycles per gate, and Grover's algorithm does not parallelize well in the way that classical SHA-256 ASIC farms do. Under generous assumptions about error-correction cycle time, a CRQC could achieve a hashrate of about 0.25 TH/s. That is more than two orders of magnitude below the 110 TH/s of a current-generation ASIC miner [3]. A separate BTQ mining-cost analysis reaches the same conclusion. In the favorable partial-preimage case, it estimates about 10^8 physical qubits and 10^4 MW. At Bitcoin's January 2025 mainnet difficulty, the estimate rises to about 10^{23} physical qubits and 10^{25} W [6]. Finally, there is no economic case for a CRQC operator to direct their hardware at mining: the marginal improvement in block-finding probability is negligible and earns standard block rewards, whereas a successful ECDSA key-derivation attack transfers existing holdings directly. ECDSA is the soft target. SHA-256 mining is not.

3 Design goals and non-goals

BTQ has six design goals. (i) NIST-standardized post-quantum signatures must be available as a first-class transaction authorization path: ML-DSA-44 (Dilithium2, FIPS 204), selected for its balance of signature size and verification speed. (ii) ECDSA and post-quantum signatures must coexist in the script interpreter through explicit opcode dispatch: `OP_CHECKSIG` and `OP_CHECKMULTISIG` retain the ECDSA path, while `OP_CHECKSIGDILITHIUM` and `OP_CHECKMULTISIGDILITHIUM` select Dilithium. Mixed-mode transactions are permitted without a hard ledger cut-over. (iii) Bitcoin's UTXO and Script model, and its witness-segregated transaction structure, are preserved without modification to their semantics. (iv) SHA-256 proof-of-work is preserved: because mining hardware is quantum-resistant at any economically plausible CRQC scale, replacing the consensus mechanism would impose costs without addressing the actual attack surface. (v) The at-rest mitigations identified by Babbush et al. are shipped as protocol defaults: BIP-360 P2MR (pay-to-Merkle-root) is supported for script-tree spending so that unused script branches never appear on-chain, and on mainnet it is where Dilithium lives: a Dilithium destination commits to a Merkle root, so neither the leaf script nor the public key appears on-chain before the spend [3, 4]. Address reuse is not consensus-prohibited. Dilithium's deterministic signing removes the nonce-reuse failure mode. Reuse still has privacy and exposure consequences, and fresh-address-per-receive remains advisable. (vi) The protocol leaves room for crypto agility through opcode allocation: additional post-quantum signature schemes can be introduced by assigning new opcode families and extending the interpreter's verification dispatch, following the signature-scheme lifecycle defined in Section 6.7. This is a deliberate hedge against the evolving post-quantum signature landscape.

Six things are explicitly outside scope. (i) BTQ is not a replacement for Bitcoin. (ii) BTQ does not migrate the existing Bitcoin ledger to post-quantum signatures. It is a clean-chain protocol with its own genesis block, which means (iii) legacy ECDSA UTXOs are not recoverable under BTQ. That boundary avoids entangling BTQ's security properties with the unresolvable key-ownership ambiguities of an existing ledger. (iv) Transaction privacy is not extended beyond what Bitcoin's base protocol offers: anonymous transactions are a distinct problem with distinct trade-offs, and conflating them with quantum resistance would obscure both. (v) Smart-contract expressiveness is not extended beyond Script: the threat model that motivates BTQ is signature-scheme weakness, not script expressiveness, and expanding the execution model would introduce orthogonal attack surface. (vi) SHA-256 proof-of-work is not replaced with a quantum-native consensus mechanism: as established in Section 2, the mining subsystem is not the vulnerable component, and introducing a novel consensus algorithm would be a risk with no corresponding threat reduction.

4 Post-quantum signatures

4.1 CRYSTALS-Dilithium

CRYSTALS-Dilithium is a lattice-based signature scheme whose security reduces to the hardness of two algebraic problems over module lattices: Module Learning With Errors (MLWE) and Module Short Integer Solution (MSIS) [8]. A forger that can produce existentially unforgeable signatures under chosen-message attack (EUF-CMA) can be converted into an algorithm solving MLWE or MSIS. No such algorithm is known. Crucially, no Shor-style polynomial-time attack against MLWE or MSIS is known. Shor's algorithm exploits the algebraic structure of cyclic groups and does not apply directly to module-lattice problems. ML-DSA is believed secure against large-scale quantum adversaries under current cryptanalytic analysis. NIST standardized Dilithium as ML-DSA in FIPS 204, published 2024-08-13 and effective 2024-08-14 [12]. BTQ bundles the pq-crystals reference C implementation, including both the pure-C reference backend and AVX2-optimized backend.

4.2 ML-DSA-44 parameters

BTQ deploys ML-DSA-44, the FIPS 204 parameter set formerly known as Dilithium2. It is NIST security category 2 (approximately AES-128 quantum security). The deployed byte sizes are: public key 1,312 bytes, secret key 2,560 bytes, and raw signature 2,420 bytes. The transaction signer appends a 1-byte sighash type to the 2,420-byte ML-DSA signature; the resulting intended non-empty stack element is 2,421 bytes. ML-DSA-44 verification is more expensive than ECDSA verification on the same hardware against the vendored pq-crystals reference backend, and key generation is correspondingly slower than secp256k1 keygen. Concrete latencies depend on CPU, compiler flags, and whether AVX2 backends are enabled. The pq-crystals reference implementation provides reproducible measurements [8].

4.3 Algorithm choice

NIST finalized two post-quantum signature standards in August 2024: ML-DSA (Dilithium, FIPS 204) and SLH-DSA (SPHINCS+, FIPS 205). FN-DSA (Falcon), the third NIST-selected signature scheme and the basis for FIPS 206, is distinct from the FIPS 204 ML-DSA standard implemented by BTQ. ML-DSA-44 is the deployed choice for the following reasons. ML-DSA-44 is NIST-standardized at security category 2 (approximately AES-128 quantum security). Its signing and verification costs are balanced, making it well-suited as a drop-in replacement for ECDSA

in a transaction-validation context where both operations occur on the hot path. Its security rests on the conservative MLWE/MSIS hardness assumption, and it ships mature reference and AVX2-optimized backends. Falcon produces smaller signatures (about 666 bytes), but its security relies on Gaussian sampling over lattices. A timing-observable bias in the Gaussian sampler can leak secret-key information, a side-channel concern noted by Babbush et al. [3]. Deterministic signing in ML-DSA avoids this class of vulnerability. SPHINCS+ / SLH-DSA is stateless and hash-based. Its security assumptions are conservative, but it produces signatures of about 8 KB and its signing is the slowest of the three. That makes it poorly suited as the primary transaction signature scheme. It remains a viable fallback if lattice hardness assumptions weaken, providing algorithm-diversity insurance. The codebase reflects this three-way assessment: integration points for Falcon and SPHINCS+ are reserved, but only ML-DSA-44/Dilithium2 is implemented. Falcon and SPHINCS+ are reserved for future integration. The current consensus implementation supports Dilithium only.

5 Consensus parameter changes

Carrying Dilithium2 signatures at consensus scale requires adjustments to four interconnected parameter groups: block weight limits, block timing and monetary policy, script element bounds, and sigops accounting.

5.1 Block weight and witness scale

BTQ raises the block capacity limits and the witness scale factor relative to Bitcoin's defaults.

- Maximum serialized block size: 8,000,000 bytes (8 MB).
- Maximum block weight: 8,000,000 weight units (8 MW).
- Witness scale factor: 16.

Bitcoin uses 4 MB / 4 MW / 4 respectively. BTQ doubles the raw byte cap and the weight cap, and quadruples the witness scale factor.

The rationale follows from the weight formula generalized to witness scale factor 16:

$$weight = base\_size \times 16 + witness\_size.$$

A witness-script Dilithium spend places the full Dilithium2 public key (1,312 bytes), the on-chain signature element (the 2,420-byte raw signature plus signature hash handling described in Section 7), and the witness script in the witness field. The non-witness portion of a typical single-input single-output transaction is about 80 bytes. Ignoring the small witness-script overhead, the dominant witness payload is about 3.7 kB. A WSF of 16 therefore makes witness data one-sixteenth as expensive in weight as non-witness data, recovering most of the per-transaction weight that PQ-sized witness fields would otherwise consume. Without the larger block cap and higher witness scale factor, the same witness payload would leave less room for additional inputs, outputs, and script overhead. The combined effect of WSF = 16 and the 8 MW cap is that PQ-sized witness data is not disproportionately penalized relative to transaction base size.

5.2 Block time, supply, and difficulty

BTQ targets 60-second block spacing, one-tenth of Bitcoin's 600-second interval. The monetary-policy constants are set to preserve Bitcoin's halving calendar at the accelerated block rate:

- Target block spacing: 60 seconds
- Initial block subsidy: 5 BTQ
- Halving interval: 2,100,000 blocks (about every four years at 60-second spacing)
- Supply cap: 21 million BTQ
- Coinbase maturity: 100 blocks

The block subsidy schedule mirrors Bitcoin's: 5 BTQ per block at launch, halving every 2,100,000 blocks, with total issuance capped at 21 million BTQ. The equivalence follows directly. Bitcoin emits 50 BTC per block on a 10-minute schedule for 210,000-block epochs. BTQ emits 5 BTQ per block on a 1-minute schedule for 2,100,000-block epochs. At 60-second spacing, 2,100,000 blocks elapse in about four years, maintaining the same nominal halving calendar.

Bitcoin's two-week difficulty-retarget cadence is retained as the pre-activation algorithm. At BTQ's 60-second target spacing this corresponds to a 20,160-block retarget interval. At launch, difficulty adjusts on the Bitcoin cadence. The linearly weighted moving average (LWMA) algorithm [16] activates at a configured post-launch block height. LWMA weights recent blocks more heavily than earlier ones within the retarget window, smoothing the adjustment response under variable hashrate. Large short-term hashrate swings produce proportionally larger timing errors within a 20,160-block window than they would on a 10-minute chain, making LWMA's faster responsiveness the appropriate long-term algorithm. LWMA is community-developed. No peer-reviewed academic reference exists [16].

5.3 Script element size and standardness

Bitcoin's script interpreter enforces a 520-byte maximum for any single stack element. The Dilithium2 public key is 1,312 bytes: 2.5× the Bitcoin limit, and therefore unable to be pushed as a script element under Bitcoin's rules.

BTQ raises this limit to 15,000 bytes. This admits both the 1,312-byte Dilithium2 public key and the 2,421-byte witness signature as direct push data within a script, with headroom for multisignature script constructions. The limit is 15 kB rather than a tighter bound because script constructions involving multiple Dilithium keys (as in P2MR leaf scripts) require stacking several key-sized elements during evaluation.

Standard transaction weight remains at Bitcoin's 400,000-weight-unit default. Dilithium-bearing transactions that approach or exceed this relay limit are non-standard unless policy is revisited.

5.4 Sigops accounting

BTQ introduces four Dilithium sigop opcodes that participate in the block-level sigops budget alongside legacy ECDSA opcodes: `0xbb` (`OP_CHECKSIGDILITHIUM`), `0xbc` (`OP_CHECKSIGDILITHIUMVERIFY`), `0xbd` (`OP_CHECKMULTISIGDILITHIUM`), and `0xbe` (`OP_CHECKMULTISIGDILITHIUMVERIFY`). The block cap is unchanged from Bitcoin:

$$\text{maximum block sigops cost} = 80{,}000.$$

Each Dilithium checksig opcode costs 50 sigops at consensus:

$$\text{Dilithium checksig cost} = 50.$$

ECDSA checksig costs 1 sigop. The ratio reflects Dilithium verification's higher per-operation CPU cost relative to ECDSA. The higher per-opcode weight prevents a block from carrying an adversarially large number of Dilithium signatures whose total verification time would make the block unacceptably expensive for validating nodes.

6 Address and script system

BTQ extends Bitcoin's address and script system along two axes: a separate address namespace for Dilithium-keyed outputs, and explicit script opcodes that accommodate lattice-based signature data. The namespace axis has since narrowed — on mainnet, Dilithium is reached exclusively through P2MR, which encodes under the classical namespace — while the opcode axis is what carries the scheme into consensus. Sections 6.1 through 6.5 cover namespace design, the pre-P2MR Dilithium templates and their present status, the witness-v2 Pay-to-Merkle-Root type, the Dilithium opcodes, and the interpreter dispatch rules.

6.1 Address-namespace design

Bitcoin encodes destinations in two formats: Base58Check addresses (P2PKH and P2SH) and Bech32/Bech32m addresses (native witness outputs). BTQ introduces Dilithium counterparts in both formats, each scoped to prevent accidental cross-algorithm sends at the encoding layer.

Legacy base58 prefixes. Bitcoin's mainnet Base58Check prefix bytes distinguish P2PKH from P2SH outputs and simultaneously prevent a Bitcoin address from decoding as a valid BTQ address. BTQ assigns distinct prefix bytes for Dilithium public-key-hash and script-hash outputs, separate from both the classical ECDSA prefix bytes and from each other. A wallet that holds a Dilithium key will produce an address that encodes differently from any ECDSA-derived address on the same network. A send to the wrong type fails at address decoding before a transaction is constructed. Testnet and regtest use their own prefix bytes following the same structure.

Two Bech32 HRPs per network. Native witness outputs are identified by a human-readable part (HRP) that prefixes the Bech32 or Bech32m encoded payload. BTQ assigns two independent HRPs at the network level, scoped by the signature type that governs spending:

- The *classical HRP* covers all witness outputs whose spending paths use ECDSA or script-only logic: P2WPKH, P2WSH, P2TR (Taproot), and P2MR (witness v2, Section 6.3). Encoding these output types under the classical HRP is consistent with Bitcoin's existing bech32/bech32m namespace.
- The *Dilithium HRP* covers Dilithium witness-v0 keyhash (P2DWPKH) addresses. The 20-byte witness-v0 keyhash program is shared with classical P2WPKH and carries no algorithm marker in the output script; the consensus verifier was selected from the size of the spending public key (Section 6.2). The Dilithium HRP was the address-layer counterpart of that distinction: an address under the Dilithium HRP cannot be decoded as a valid classical-HRP address, so a wallet encoded a Dilithium-spendable keyhash destination distinctly from an ECDSA one even though the two share a witness-program shape on-chain. That distinction did not prove sufficient, and the witness-v0 Dilithium path has since been withdrawn (Section 6.2). The HRP remains defined at the network level so that testnet history stays decodable, but it has no output type that is spendable on mainnet, and the wallet generates no addresses under it.

The address encoder selects between the two HRPs based on destination type. On mainnet every

Dilithium destination the wallet generates is a P2MR output, and therefore encodes under the classical HRP as described next.

P2MR placement note. Pay-to-Merkle-Root (Section 6.3) encodes under the classical HRP even though its leaf scripts may contain Dilithium opcodes. The address commits to a 32-byte Merkle root of a script tree and carries no information about which leaf scripts are present or which signature type satisfies them. Because the address does not bind the spending algorithm, it belongs in the classical HRP namespace. The quantum-resistance property emerges from the leaf script, not the address prefix.

This dual-HRP structure is forward-extensible: future witness versions or address types for Falcon, SPHINCS+, or another signature scheme could be assigned to additional HRPs within the Dilithium namespace, or to a third HRP, without colliding with the classical namespace or the existing Dilithium entries.

6.2 Pay-to-Dilithium scripts

Dilithium verification is reached through scripts that carry explicit Dilithium opcodes, and the script version in which those opcodes are consensus-valid is itself a height-gated consensus rule (Section 6.6). Two regimes therefore have to be distinguished, and they differ by network.

Under the *P2MR-only* regime, Dilithium opcodes are consensus-valid only inside P2MR tapscript leaves (Section 6.3); the same opcodes in a legacy script, a P2SH redeem script, or a witness-v0 script are rejected, and the witness-v0 keyhash program no longer routes to Dilithium at all. This is the regime BTQ mainnet enforces from genesis. P2MR leaf scripts are consequently the only path by which Dilithium reaches consensus on mainnet, and the wallet creates no other kind of Dilithium destination.

Under the earlier *pre-P2MR* regime, Dilithium opcodes were additionally valid in legacy scripts, P2SH, and witness-v0 scripts, and a Dilithium-sized public key on a witness-v0 keyhash program routed to the Dilithium verifier. That regime remains in force on the public testnet, whose chain history contains Dilithium outputs of these earlier shapes, and it is the reason the templates below are described here at all. The remainder of this subsection documents those templates: the leaf-script forms carry forward into P2MR, and the output-level forms are retained as testnet history rather than as mainnet destinations.

P2DPK (Pay-to-Dilithium-PubKey). The locking script embeds the full Dilithium2 public key directly:

```
<1312-byte Dilithium2 pubkey> OP_CHECKSIGDILITHIUM
```

This pattern is the quantum analogue of Bitcoin's P2PK: the public key is committed on-chain at output creation, and the unlocking data need only supply a valid signature. This paper recommends against P2DPK for long-held funds. Because the public key is visible in the UTXO set from the moment the output is created, it satisfies the Weak Address definition under the Aggarwal–Babbush vulnerability classification (Section 2.2): a sufficiently capable quantum adversary can derive the corresponding private key before the output is spent. Under the P2MR-only regime this shape survives as a P2MR leaf script — a single-key leaf committing one Dilithium public key — where the Merkle root hides it until the spend, so the Weak Address exposure above does not arise. As a bare output script it belongs to the pre-P2MR regime: on mainnet such an output is not spendable, which is why single-key P2MR, and not P2DPK, is the recommended coinbase payout target.

Dilithium public-key-hash scripts. A Dilithium public-key-hash template commits to the 20-byte HASH160 of the Dilithium public key and checks a Dilithium signature at spend time:

```
OP_DUP OP_HASH160 <20-byte HASH160(Dilithium2 pubkey)> OP_EQUALVERIFY
OP_CHECKSIGDILITHIUM
```

The unlocking data carries the signature and the full public key. This gives the same address-hash protection as Bitcoin's P2PKH family: the output exposes only a hash before it is spent, and the public key appears when the spend is broadcast. Used directly or wrapped behind a script hash, this template belongs to the pre-P2MR regime; the corresponding Base58 Dilithium address format is retained for reading testnet history and is no longer generated.

Witness-script Dilithium spends. Witness-v0 script-hash outputs can commit to a witness script that contains `OP_CHECKSIGDILITHIUM` or `OP_CHECKMULTISIGDILITHIUM`. At spend time the witness stack supplies the Dilithium signature material and the witness script; the opcode in that script selects the Dilithium verifier. This is the witness-script route by which Dilithium reached the interpreter before P2MR. Under the P2MR-only regime the same multi-key policies are expressed as P2MR leaf scripts instead, either with `OP_CHECKMULTISIGDILITHIUM` directly or as a threshold accumulator built from `OP_CHECKSIGDILITHIUM` and `OP_ADD`.

P2DWPKH (witness-v0 keyhash). A witness-v0 keyhash program is a 20-byte witness program that commits to the HASH160 of a public key, with the spending public key supplied on the witness stack. The output script is identical for ECDSA and Dilithium: it carries no opcode that names the algorithm. The interpreter therefore selects the verifier from the size of the witness public key. A 33-byte compressed key takes the ECDSA path, reconstructing the `OP_DUP OP_HASH160 <program> OP_EQUALVERIFY OP_CHECKSIG` script of Bitcoin's P2WPKH. A 1,312-byte Dilithium2 public key reconstructs the same script with `OP_CHECKSIGDILITHIUM` in place of `OP_CHECKSIG` and verifies under Dilithium. This is the witness-v0 keyhash (P2DWPKH) path: it gives the same address-hash protection as P2WPKH under a Dilithium key, exposing only a 20-byte hash before the spend.

This path has been withdrawn. It is the one place where the spending algorithm was inferred rather than named, and an output script that does not say which algorithm governs it is too easily confused with the classical P2WPKH it is byte-identical to. Under the P2MR-only regime the size heuristic is disabled: a Dilithium-sized key in a witness-v0 keyhash spend falls through to the ECDSA path and the spend is rejected. Mainnet enforces this from genesis and the wallet no longer creates P2DWPKH destinations; an output of this shape can still be paid to, but it cannot be spent under Dilithium. The path is described here because it remains live in testnet history, and because its withdrawal is what leaves every Dilithium spend on mainnet named by an explicit opcode.

6.3 Witness v2 / BIP-360 Pay-to-Merkle-Root (P2MR)

BTQ implements BIP-360 P2MR [4] (Beast, Heilman, and Foxen Duke, Draft proposal in Bitcoin) as a witness version 2 output type. P2MR is a Taproot variant that removes key-path spending entirely, retaining only script-path spends. The scriptPubKey commits to a 32-byte Merkle root of a script tree. The spender supplies a control block, the satisfied leaf script, and the corresponding witness data. No public key appears on-chain until the output is spent: the UTXO is a 32-byte Merkle root and nothing more. This design follows the recommendation of Babbush et al. §III.A, which identifies a Taproot-style output with no key-path spend as the correct mitigation for the Weak Address vulnerability in a post-ECDSA network [3].

Scope of the mitigation. P2MR eliminates the Weak Address vulnerability that re-emerges with P2TR. See Section 2.2, Aggarwal §3 [1], and Babbush §III.A [3]. Because the UTXO exposes only a Merkle root, a slow-clock cryptographically-relevant quantum computer (CRQC) cannot derive a spending key from on-chain data while the output sits unspent. P2MR does *not* by itself eliminate Public Mempool Exposure. When a P2MR output is spent, the leaf script and any signatures it contains become visible in the mempool before block inclusion, recreating the short-window exposure that Babbush et al. analyse for signature-bearing spends. P2MR is therefore a complete answer to at-rest attacks (slow-clock CRQCs) and a partial answer to on-spend attacks. The residual on-spend window is closed by using post-quantum signatures inside the leaf script. BTQ does this by enabling Dilithium opcodes (`0xbb`–`0xbf`) inside P2MR leaf scripts and blocking them inside P2TR leaves (Section 6.4). P2TR thereby remains Bitcoin-compatible. P2MR is the quantum-resistant script-tree extension surface.

Address encoding. P2MR outputs encode as bech32m addresses under the classical HRP (Section 6.1), with witness version 2 and a 32-byte Merkle root payload (`<network-hrp>1z...`-style). The encoder selects the classical HRP for this destination type.

Consensus enforcement. P2MR verification is enabled unconditionally in block validation. The interpreter checks the Merkle commitment, enforces the parity bit, and parses the control block. The expected witness program size is 32 bytes. P2MR leaf execution uses a dedicated script version in the interpreter so that Dilithium opcodes are enabled there while remaining blocked inside ordinary P2TR leaves.

Policy. Standardness accepts witness v2 P2MR outputs.

Wallet and RPC surface. BTQ ships a dedicated P2MR vault flow. The RPC surface includes address creation (`getnewp2mraddress`), funding (`sendtop2mr`), listing (`listp2mr`), and inspection (`getp2mrinfo`). It also includes P2MR spend construction (`createp2mrspend`), signing (`signp2mrtransaction`), and testing (`testp2mrtransaction`). P2MR UTXOs are tracked by explicit scriptPubKey equality rather than ordinary wallet ownership inference. Standard coin selection does not sweep P2MR funds. Spends use the P2MR-specific path. This isolation preserves clean accounting semantics and prevents unintentional consumption of quantum-protected outputs by a classical spending path.

Status. BIP-360 is a Draft proposal in Bitcoin [4]. Bitcoin Core has not adopted it. BTQ ships P2MR consensus enforcement, wallet infrastructure, RPC support, and tests.

6.4 Dilithium opcodes

BTQ allocates opcodes through `OP_DILITHIUM_PUBKEY` (`0xbf`), extending Bitcoin's script opcode range beyond `OP_CHECKSIGADD` (`0xba`). This is a meaningful divergence from Bitcoin's script-interpreter specification: any implementation that enforces the original Bitcoin opcode range will reject BTQ transactions that use the five opcodes below.

Stack semantics for CHECKSIG variants. Both `OP_CHECKSIGDILITHIUM` and `OP_CHECKSIG DILITHIUMVERIFY` consume `<sig>` and `<pubkey>` from the stack and return 1 or 0 (or abort with the VERIFY variant on failure). A non-empty signature element is required to be exactly 2,421 bytes: a 2,420-byte Dilithium2 signature followed by a 1-byte sighash type. An empty signature element is permitted and evaluates to a failed check, which combined with `NULLFAIL` enforcement means a non-empty element that fails verification aborts the script.

Opcode	Hex	Function
OP_CHECKSIGDILITHIUM	0xbb	Verify single Dilithium signature
OP_CHECKSIGDILITHIUMVERIFY	0xbc	As above, fail on invalid
OP_CHECKMULTISIGDILITHIUM	0xbd	M-of-N Dilithium multisig
OP_CHECKMULTISIGDILITHIUMVERIFY	0xbe	As above, fail on invalid
OP_DILITHIUM_PUBKEY	0xbf	Validate Dilithium public-key shape

Table 2: Dilithium opcodes added to the BTQ script interpreter.

Verification process. The interpreter extracts the sighash type from the final byte of the signature element, computes the transaction sighash per BIP-143 [10], and verifies the first 2,420 bytes against the supplied public key and the computed sighash using the Dilithium2 verify routine.

Multisig. `OP_CHECKMULTISIGDILITHIUM` and its `OP_CHECKMULTISIGDILITHIUMVERIFY` variant implement opcode-level M-of-N by sequentially verifying multiple independent Dilithium signatures. True threshold or aggregated lattice signatures are out of scope for the current protocol.

`OP_DILITHIUM_PUBKEY`. This opcode is executed when present in a script. It takes a single stack element, checks that the element is a structurally valid Dilithium public key, and pushes the boolean result. It is used in templates that need to assert Dilithium public-key shape without performing a signature check.

6.5 Explicit-opcode dispatch in the interpreter

The interpreter selects the signature verifier from the opcode, not from the byte length of the public key. `OP_CHECKSIG` and `OP_CHECKMULTISIG` retain the ECDSA verification path in base and witness-v0 script. `OP_CHECKSIGDILITHIUM` and `OP_CHECKMULTISIGDILITHIUM` route to the Dilithium verifier. The selected path then enforces the expected signature and public-key shape for that algorithm.

This is the mechanism that makes BTQ a dual-algorithm protocol at the script layer. ECDSA remains available where the script asks for ECDSA. Dilithium is available where the script asks for Dilithium. The script template chooses the algorithm by choosing the opcode. Higher layers can build mixed-mode transactions because the interpreter carries both opcode families at once.

The witness-v0 keyhash program was the one exception to opcode-driven dispatch. That program type commits only to a 20-byte hash and supplies the public key on the witness stack, so the output script contains no opcode naming the algorithm, and for this program type alone the interpreter selected the verifier from the size of the witness public key: a compressed-secp256k1-sized key took the ECDSA path, and a Dilithium2-public-key-sized key took the Dilithium path. That exception has been withdrawn (Section 6.2). Where the P2MR-only rule is in force — on mainnet, from genesis — the size heuristic is disabled and dispatch is opcode-driven without exception.

6.6 Dilithium activation

Two buried deployments govern Dilithium, and they do different jobs. The first gates verification: the interpreter applies Dilithium verification behaviour only once its activation height is reached, and before that height a Dilithium public-key type is treated as an unrecognised, discouraged public-key type rather than verified, leaving the rule upgradable. The second gates *scope*: once its activation height is reached, the Dilithium opcodes are consensus-valid only inside P2MR tapscript leaves, and

the witness-v0 keyhash size heuristic that routed a spend to Dilithium is disabled (Section 6.2). A deployment height can therefore restrict where an opcode family may appear, not only whether it is verified.

Both heights are network parameters. BTQ mainnet sets both to genesis, so mainnet has never had a pre-activation window and has only ever enforced the P2MR-only regime. The public testnet has a scheduled verification height and pre-P2MR history, and its scope height is deliberately left unscheduled so that the legacy Dilithium outputs already in its chain remain spendable; upgraded nodes decline to relay such spends as a policy matter without treating peers that still relay them as misbehaving. Scheduling the scope deployment retroactively would invalidate that history, which is why the restriction is height-gated rather than applied unconditionally.

P2MR verification (Section 6.3), by contrast, is enabled unconditionally. Gating Dilithium as buried deployments, in the same manner as the LWMA difficulty transition (Section 5.2), lets a network reach a known height with consistent validation behaviour before post-quantum spending semantics become mandatory.

6.7 Signature-scheme lifecycle

BTQ defines a lifecycle by which a signature scheme enters consensus, and ML-DSA-44 is the first scheme carried through it. A scheme is identified by its opcode family: verification opcodes are allocated per scheme (for ML-DSA-44, `OP_CHECKSIGDILITHIUM`, `OP_CHECKMULTISIGDILITHIUM`, and their variants), and the interpreter dispatches on the opcode (Section 6.5); the witness-v0 keyhash program is the one template that carries no such opcode and is disambiguated by public-key size instead. A scheme becomes spendable through height-gated activation (Section 6.6); before its activation height, its public-key types are treated as unrecognised and discouraged rather than verified, leaving the rule upgradable. Height gating also scopes where a scheme's opcodes are consensus-valid, not only when: a deployment height can confine an opcode family to a particular script version.

This lifecycle is the protocol's hedge against the youth of post-quantum cryptanalysis. The lattice assumptions that secure ML-DSA-44 today may weaken, and stronger schemes may emerge. A protocol that freezes one signature algorithm into consensus inherits the lifetime risk of that algorithm's assumptions. Registering a successor scheme under this lifecycle consists of vendoring the primitive, allocating an opcode family, extending the interpreter's verification dispatch, and scheduling an activation height: consensus work requiring implementation, review, and audit, but confined to additions within the Script execution model rather than a redesign of it. Three components remain per-scheme and are documented future work: address and witness-program encoding, sigops and weight accounting for the new scheme's key and signature sizes, and wallet key management.

7 Sighash and signing

BTQ follows Bitcoin's sighash model for Dilithium scripts. For witness-v0 scripts, the preimage construction is the ten-field BIP-143 serialization, with the `scriptCode` field containing the Dilithium witness script [10]. The hash is computed as `SHA256d(preimage)`, Bitcoin's double SHA-256. The transaction signer appends the sighash type to the raw Dilithium signature as a 1-byte trailer, producing a 2,421-byte stack element for a normal non-empty signature.

All four Bitcoin sighash types are supported: `SIGHASH_ALL` (0x01, sign all inputs and all outputs),

SIGHASH_NONE (0x02, sign all inputs and no outputs), SIGHASH_SINGLE (0x03, sign all inputs and the output at the same index), and the SIGHASH_ANYONECANPAY modifier (0x80, combined with any of the above to permit additional inputs to be added after signing). These carry the same semantics as in Bitcoin.

Worked example: P2WSH-style Dilithium spend with SIGHASH_ALL

The following shows the full sighash preimage construction for a single-input witness-script Dilithium spend using SIGHASH_ALL. The field layout follows BIP-143. The scriptCode is the Dilithium public-key-hash witness script.

```
--- sighash preimage (BIP-143 layout, SIGHASH_ALL) ---

nVersion        4 bytes  (little-endian int32)
hashPrevouts   32 bytes  SHA256d of all outpoints (SIGHASH_ALL: all inputs)
hashSequence   32 bytes  SHA256d of all nSequence values (SIGHASH_ALL)
outpoint       36 bytes  txid (32) || vout (4) of the input being signed
scriptCode     25 bytes  OP_DUP OP_HASH160 <20-byte HASH160> OP_EQUALVERIFY
                           OP_CHECKSIGDILITHIUM
amount          8 bytes  value of the UTXO being spent (little-endian int64)
nSequence       4 bytes  nSequence of the input being signed
hashOutputs    32 bytes  SHA256d of all outputs serialized (SIGHASH_ALL)
nLockTime       4 bytes  (little-endian uint32)
sighashType     4 bytes  0x01000000 (SIGHASH_ALL, little-endian uint32)
```

The sighash is then:

$$sighash = \text{SHA256d}(preimage)$$

The wallet signs with Dilithium2 and appends the sighash type byte:

```
sig_raw  = Dilithium2.Sign(sk, sighash)   -- 2,420 bytes
sig_wire = sig_raw || 0x01                -- 2,421 bytes (SIGHASH_ALL trailer)
```

The witness stack placed on-chain for this spend is:

```
witness[0]  <2421-byte sig_wire>
witness[1]  <1312-byte Dilithium2 pubkey>
witness[2]  <witnessScript>
```

The interpreter recovers the sighash type from the final byte of witness[0], recomputes the preimage, hashes it, and calls Dilithium2.Verify(pubkey, sighash, sig_raw). Sighash semantics are otherwise unchanged from SegWit v0. Only the final verification step differs: Dilithium2.Verify replaces secp256k1.Verify. The binding between sighash and signature algorithm is established at the opcode layer (Section 6.4) rather than in the sighash field itself.

8 Wallet integration

8.1 Key types and identifiers

BTQ stores an expanded in-memory Dilithium keypair as 3,872 bytes: a 2,560-byte Dilithium2 secret key followed by the 1,312-byte public key. The HD parent object carries a 73-byte serialisation

(1-byte depth, 4-byte fingerprint, 4-byte child index, 32-byte chaincode, 32-byte seed), while leaf keys are stored as expanded Dilithium key material. The seed and chaincode determine child keys; the leaf key is the expanded signing object used by the wallet. A public key's identifier is computed as KeyID = RIPEMD160(SHA256(pk)), a 20-byte value identical in structure to Bitcoin's `HASH160(pubkey)`. This identity is what gives Dilithium public-key-hash destinations the same 20-byte identifier shape as Bitcoin's public-key-hash destinations.

8.2 Wallet types

Both the legacy wallet path and the descriptor-wallet path handle Dilithium destinations. Address-generation surfaces produce Dilithium public-key-hash and witness-v0 key-hash destinations. Wallet and script plumbing also recognise raw Dilithium public-key destinations, Dilithium script-hash destinations, witness-v0 script-hash destinations, and P2MR destinations.

8.3 HD derivation: hardened-only, shipped

BTQ ships hardened-only HD derivation for Dilithium keys. The serialised extended private key is 73 bytes and the serialised extended public key is 1,353 bytes. External keys use derivation path $m/0'/0'/n'$. Change keys use $m/0'/1'/n'$.

Non-hardened (xpub-style) derivation is absent. Elliptic-curve public keys support homomorphic tweak operations: a child public key can be derived from a parent public key without the private key, which is the mechanism that powers BIP32 watch-only wallets. Dilithium public keys, as structured lattice objects, do not admit this operation. BTQ refuses non-hardened indices rather than silently producing an unrestorable keypair. Deegan, Fitzwater, Gur, and Nugent (Project Eleven) confirm that recovering xpub-style derivation in the post-quantum setting is an open problem [7]. BTQ's hardened-only design is the principled response: it preserves backup-from-seed user experience while sidestepping the non-tweakability obstacle.

8.4 Storage and encryption

Dilithium key material is encrypted with AES-256-CBC under the wallet master key, using the same code path as Bitcoin Core's secp256k1 key encryption. No novel cryptography is introduced at this layer. The HD parent serialisation is the 73-byte extended private key described in Section 8.1; generated and imported leaf keys use the expanded Dilithium key material needed for signing. Both numbers are larger than their secp256k1 counterparts (32-byte raw key, 74-byte BIP32 extkey), so backup payloads and per-key memory footprints grow accordingly. This is consequential for backups but not for runtime operation on modern hardware, and no special handling is required.

8.5 RPC surface

Five Dilithium wallet RPC methods are exposed:

A Dilithium-aware private-key export command (`dumpdilithiumkey`) is not exposed by this RPC surface.

Method	Purpose
`getnewdilithiumaddress`	Derive a fresh Dilithium-spendable address
`signmessagewithdilithium`	Sign an arbitrary message with a Dilithium key
`verifydilithiumsignature`	Verify a Dilithium signature against an address and message
`signtransactionwithdilithium`	Sign a raw transaction using Dilithium keys
`importdilithiumkey`	Import a Dilithium private key into the wallet

Table 3: Dilithium wallet RPC methods.

9 Network parameters

9.1 Codebase fork, separate ledger

BTQ is a codebase fork of Bitcoin Core [5], maintained in the project repository `btq-ag/btq-core`. BTQ operates as a *separate, replay-isolated blockchain* with its own genesis block, magic bytes, and address namespace. BTQ is not a layer built on top of Bitcoin.

BTQ inherits Bitcoin Core's protocol architecture, peer-to-peer networking stack, and battle-tested block-validation logic, while running a fresh chain whose consensus parameters are set from scratch to accommodate post-quantum signature sizes. Three concrete advantages follow from the clean-chain choice:

1. No Bitcoin transaction replay risk: the distinct genesis and magic bytes ensure BTQ packets are rejected by Bitcoin nodes and vice versa.
2. No Bitcoin ECDSA UTXO migration: BTQ starts from its own genesis block, and Bitcoin UTXOs do not carry over to the BTQ ledger.
3. Full freedom to tune block-weight accounting, script limits, and the difficulty algorithm without backward-compatibility constraints imposed by the existing Bitcoin UTXO set.

At the referenced release, the public deployment is testnet.

9.2 Address-namespace and chain-identity design

The structural design of BTQ's address namespace is described in Section 6.1: dual human-readable parts distinguishing pay-to-Dilithium addresses from classical pay-to-script-hash paths, and a distinct version-byte namespace that prevents any address from being valid on both the BTQ chain and the Bitcoin mainnet or testnet.

Current deployment constants, including HRPs, prefix bytes, magic bytes, default port numbers, genesis hash, and coinbase commitment strings, are maintained in release documentation and are not finalized until network activation.

9.3 Monetary policy

BTQ preserves Bitcoin's monetary-supply design:

- **Hard supply cap of 21 million BTQ.** The cap is enforced by the subsidy schedule, identical in structure to Bitcoin's: a fixed initial block reward that halves at a fixed block-count interval, converging to zero over time.

- **About four-year halving calendar.** Because BTQ targets 60-second blocks, one-tenth Bitcoin's interval, the halving interval is set to 2,100,000 blocks so that the nominal calendar duration matches Bitcoin's halving schedule (see Section 5.2 for the derivation).
- **No premine, no pre-allocated supply.** The genesis block carries only the coinbase reward. No supply is allocated to developers, a foundation, or any other party outside the open block-production process.

The exact deployment values are maintained in release documentation.

9.4 Difficulty algorithm

Block difficulty is regulated by the Linearly Weighted Moving Average (LWMA) algorithm [16], targeting a 60-second block interval. LWMA was selected over Bitcoin's legacy two-week retarget window because it responds to rapid hash-rate changes within a few windows rather than over weeks. This gives faster response during early network formation, when the mining population may grow or contract quickly.

LWMA activates from a post-launch checkpoint height. The specific activation height is a deployment constant and is not enumerated here.

10 Security analysis

10.1 Reduction to MLWE/MSIS

BTQ's signature scheme inherits Dilithium's EUF-CMA (existential unforgeability under chosen-message attack) security reduction [8, 12]. Under the Module-LWE (MLWE) and Module-SIS (MSIS) assumptions, which are instances of the short-integer solution and learning-with-errors problems over polynomial rings, an adversary who forges a Dilithium signature can be converted, via a tight reduction, into an algorithm that solves the corresponding module-lattice problem. The NIST security level for Dilithium2 is Category 2 ($\geq$ 128-bit post-quantum). The exact parameter analysis appears in Section 4.2.

The reduction covers mathematical unforgeability of the signature scheme itself. It does not cover:

- *Implementation correctness* of the vendored pq-crystals implementation or its integration with the Bitcoin Script interpreter.
- *Side-channel resistance*: the proof is in the random-oracle model and says nothing about power-analysis, timing, or cache-access traces on physical hardware.
- *Randomness-source quality*: Dilithium2 uses deterministic signing (deriving the per-signature nonce from the message and key via a PRF), which eliminates the catastrophic failure mode of reusing r in ECDSA, but the security of the PRF still depends on the quality of the underlying pseudorandom-function implementation.

10.2 What BTQ does and does not protect

Aggarwal et al. [1] and Babbush et al. [3] identify four vulnerability classes for ECDSA-based blockchains under quantum attack (Section 2.2). The following maps each class to BTQ's coverage, using the address types defined there.

Weak address. Eliminated for the recommended address patterns. P2MR (Section 6.3) likewise encodes no public key in the output script. On mainnet this holds by construction rather than by convention: every Dilithium destination is a P2MR output, whose script commits to a Merkle root and therefore withholds both the leaf script and the public key until the spend. The pre-P2MR templates achieved the same effect where they committed to a script hash or a public-key hash, with bare P2DPK as the exception — it places the public key on-chain at output creation, and this paper recommends against it for long-held funds (Section 6.2).

Address reuse. Address reuse is not consensus-prohibited. Dilithium2 signing is stateless and deterministic, so reuse does not create a nonce-reuse failure mode. After the first spend, the public key is on-chain. Reuse still links payments and increases public-key exposure. Fresh addresses remain the wallet practice.

Public mempool exposure. This is the residual vulnerability class for BTQ. Spending a hash-committed Dilithium UTXO requires broadcasting the public key or the leaf script that contains it before the transaction is included in a block. This is the same exposure pattern as Bitcoin SegWit. A sufficiently powerful quantum adversary observing the mempool could, in principle, derive the private key from the public key during the confirmation window and submit a conflicting transaction.

Dilithium2's MLWE/MSIS hardness pushes the estimated time-to-break far beyond what is achievable with any quantum computer described in the current literature, making this attack impractical against Dilithium2 under present threat models. The mempool-window argument is nonetheless identical to that for ECDSA. The improvement is quantitative, not architectural. Stewart et al.'s commit-delay-reveal proposal [15] would close this residual class for Bitcoin-derived protocols. BTQ does not implement commit-delay-reveal.

Offchain exposure. User-practice concern. BTQ inherits Bitcoin's situation: private keys stored or transmitted insecurely outside the chain are outside the scope of any on-chain cryptographic guarantee.

Legacy ECDSA UTXOs (out of scope). This concern is distinct from the four vulnerability classes above and does not apply to BTQ. BTQ is a clean-chain fork with no ECDSA UTXOs. The dormant-asset policy problem on Bitcoin proper is out of scope for this document. That problem is how to handle legacy outputs that become spendable by a quantum adversary. Babbush et al. [3] §VIII provide the relevant policy analysis for that setting.

10.3 Migration risks

Babbush et al. [3] §VII identify structural challenges that any PQ migration of a deployed blockchain must confront. BTQ inherits several of them.

Post-quantum cryptography has faced less public scrutiny than the ECDLP-based protocols it replaces. ECDSA has three decades of cryptanalytic attention behind it, while Dilithium's security track record, although accelerated by the NIST competition, is shorter. PQ signatures are one to two orders of magnitude larger than ECDSA signatures. BTQ's response is the combined effect of witness scale factor 16 and the 8 MB block cap (Section 5.1), which prevents lattice-sized witness data from consuming disproportionate weight. Lattice signing has a history of side-channel surfaces, particularly in Gaussian-sampling implementations. Falcon's floating-point Gaussian sampler is the

canonical example. BTQ's selection of Dilithium2, which uses rejection sampling and deterministic signing (Section 4.3), avoids the Falcon-specific failure mode, but inherits any future side-channel finding against the pq-crystals implementation or the MLWE/MSIS assumptions themselves.

10.4 Operational risks

DoS economics under WSF = 16. Changing the witness scale factor alters the cost structure for DoS attempts that target witness-heavy transactions. WSF = 16 reduces the weight charged per witness byte. An adversary can therefore push more witness data per weight unit than under Bitcoin's WSF = 4. The tradeoff is analyzable under standard CPU-per-weight-unit framing (Section 5.1): the relevant question is whether the CPU cost of validating a Dilithium signature per WU consumed increases or decreases relative to Bitcoin's ECDSA baseline.

10.5 Comparison to other post-quantum blockchains

Babbush et al. [3] §VI.C survey the existing landscape of post-quantum blockchain deployments. The principal projects in this space are:

- *QRL* (2018): launched on XMSS (stateful hash-based signatures). A migration to ML-DSA (Dilithium) is underway.
- *Mochimo*: transaction signatures based on WOTS+ (stateful, one-time signatures), requiring address-state management by the wallet.
- *Abelian*: lattice-based protocol with integrated privacy primitives.
- *Algorand*: Falcon-based state proofs were introduced in 2022; Babbush et al. report the first PQC-secured transaction in 2025 [2, 3]. ECDSA remains in use for transaction-level signatures.
- *Solana, XRPL*: experimental post-quantum deployments on test instances, not yet in production.

BTQ is structurally distinct from all of the above: it is a UTXO-model Bitcoin Core fork with explicit Dilithium transaction-signature opcodes alongside the retained ECDSA script surface, and no account-model redesign. This comparison is limited to the projects surveyed in Babbush §VI.C [3] and the public project documentation cited here. Babbush et al. do not endorse or evaluate Bitcoin-chain-fork approaches. The positioning above reflects BTQ's own characterization of its design space.

A Reference implementation locations

The main text describes the BTQ protocol. The reference implementation is `btq-ag/btq-core` [5]. The table below maps each protocol claim to the area of the reference implementation where the corresponding mechanism is defined.

Protocol claim	Value or mechanism	Implementation area
Signature scheme	ML-DSA-44 / Dilithium2; 1,312-byte public key, 2,560-byte secret key, 2,420-byte raw signature	`src/crypto/dilithium_wrapper.h`; `src/crypto/dilithium/`

Block capacity and witness scale	8 MB serialized block cap; 8 MW block-weight cap; witness scale factor 16	`src/consensus/consensus.h`
Monetary policy and timing	60-second target spacing; 5 BTQ initial subsidy; 2,100,000-block halving interval; 21 million BTQ cap; 100-block coinbase maturity	`src/kernel/chainparams.cpp`
Difficulty adjustment	Bitcoin retarget before activation; LWMA after the configured activation height	`src/pow.cpp`; `src/kernel/chainparams.cpp`
Script element and transaction limits	15,000-byte maximum script element; 400,000-weight-unit standard transaction limit	`src/script/script.h`; policy limit definitions
Sigops accounting	80,000 maximum block sigops cost; Dilithium checksig cost 50	`src/consensus/consensus.h`; `src/script/script.h`; `src/script/script.cpp`
Address namespaces	Distinct Dilithium legacy prefixes; separate classical and Dilithium Bech32 HRPs	`src/kernel/chainparams.cpp`; `src/key_io.cpp`
P2MR support	Witness version 2; 32-byte Merkle-root output; consensus commitment verification	`src/addresstype.h`; `src/script/interpreter.cpp`; `src/validation.cpp`
Dilithium opcodes	`0xbb`–`0xbf`: Dilithium checksig, multisig, and public-key validation opcodes	`src/script/script.h`; `src/script/interpreter.cpp`
Sighash and dispatch	Explicit Dilithium opcode dispatch; pre-P2MR witness-v0 keyhash dispatch by public-key size, disabled under the P2MR-only rule; signer appends sighash byte	`src/script/sign.cpp`; `src/script/interpreter.cpp`
Dilithium activation	Two height-activated buried deployments: one gating Dilithium verification, one confining the Dilithium opcodes to P2MR tapscript and disabling witness-v0 Dilithium routing. Both at genesis on mainnet; `P2MR` enabled unconditionally	`src/consensus/params.h`; `src/validation.cpp`; `src/kernel/chainparams.cpp`
Wallet integration	Hardened-only Dilithium HD derivation; Dilithium destination types; wallet RPC surface	`src/crypto/dilithium_key.h`; `src/crypto/dilithium_key.cpp`; `src/wallet/scriptpubkeyman.cpp`; `src/wallet/rpc/dilithium.cpp`

References

[1] Divesh Aggarwal, Gavin K. Brennen, Troy Lee, Miklos Santha, and Marco Tomamichel. Quantum attacks on bitcoin, and how to protect against them. *Ledger*, 3, 2018. URL `https://arxiv.org/abs/1710.10377`.

[2] Algorand Foundation. Algorand's post-quantum blockchain technology, 2026. URL https://algorand.co/technology/post-quantum.

[3] Ryan Babbush, Adam Zalcman, Craig Gidney, Michael Broughton, Tanuj Khattar, Hartmut Neven, Thiago Bergamaschi, Justin Drake, and Dan Boneh. Securing elliptic curve cryptocurrencies against quantum vulnerabilities: Resource estimates and mitigations. *arXiv preprint*, April 2026. URL https://arxiv.org/abs/2603.28846.

[4] Hunter Beast, Ethan Heilman, and Isabel Foxen Duke. BIP-360: Pay-to-merkle-root (P2MR), 2024. URL https://github.com/bitcoin/bips/blob/master/bip-0360.mediawiki. Status: Draft.

[5] BTQ Technologies. BTQ Core reference implementation, 2026. URL https://github.com/btq-ag/btq-core.

[6] Pierre-Luc Dallaire-Demers and BTQ Technologies Team. Kardashev scale quantum computing for bitcoin mining. *arXiv preprint*, March 2026. doi: 10.48550/arXiv.2603.25519. URL https://arxiv.org/abs/2603.25519.

[7] Conor Deegan, James Fitzwater, Kamil Doruk Gur, and David Nugent. Lattice HD wallets: Post-quantum BIP32 hierarchical deterministic wallets from lattice assumptions. IACR ePrint 2026/380, February 2026. URL https://eprint.iacr.org/2026/380.

[8] Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Dilithium: A lattice-based digital signature scheme. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2018(1), 2018. URL https://eprint.iacr.org/2017/633.

[9] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing (STOC)*, pages 212–219, 1996.

[10] Johnson Lau and Pieter Wuille. BIP-143: Transaction signature verification for version 0 witness program, 2016. URL https://github.com/bitcoin/bips/blob/master/bip-0143.mediawiki. Status: Deployed.

[11] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, 2008. URL https://bitcoin.org/bitcoin.pdf.

[12] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Technical Report FIPS 204, NIST, August 2024. URL https://nvlpubs.nist.gov/nistpubs/fips/NIST.FIPS.204.pdf.

[13] Project Eleven. Bitcoin risq list, 2026. URL https://www.projecteleven.com/bitcoin-risq-list. Accessed 2026-05-30. Methodology repository: https://github.com/p-11/bitcoin-risq-list-query.

[14] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.

[15] Iain Stewart, Daniel Ilie, Alexei Zamyatin, Sam M. Werner, M. F. Torshizi, and William J. Knottenbelt. Committing to quantum resistance: a slow defence for Bitcoin against a fast quantum computing attack. *Royal Society Open Science*, 5(6):180410, June 2018. URL https://royalsocietypublishing.org/doi/10.1098/rsos.180410.

[16] zawy12. LWMA difficulty algorithm, 2017. URL `https://github.com/zawy12/difficulty-algorithms`. Community-developed; no peer-reviewed academic reference exists. Issues #3 and #24.